

CSF 性能瓶颈与核心代码分析报告

1. 核心代码认定

1.1 核心代码文件: `Cloth.cpp` + `Particle.cpp`

理由:

- `Cloth.cpp` 包含布料模拟的全部核心逻辑: 物理迭代循环 (`timeStep`)、地形碰撞 (`terrCollision`)、坡度平滑后处理 (`movableFilter`)
- `Particle.cpp` 包含每次迭代中计算量最大的两个操作: Verlet积分 (`timeStep`) 和约束满足 (`satisfyConstraintSelf`)
- 两者构成了算法最关键的"模拟循环", 占据了总执行时间的绝大部分

1.2 辅助核心文件: `Rasterization.cpp`

- 光栅化阶段是模拟前的数据准备, 计算复杂度虽为 $O(N)$, 但对大数据集不可忽视
- 空洞填充的扫描线/BFS搜索在最坏情况下可达 $O(W \times H)$

1.3 非核心文件

文件	原因
<code>Constraint.cpp/h</code>	虽然存在, 但未被实际使用。约束逻辑已被内联到 <code>Particle::satisfyConstraintSelf</code> 中
<code>XYZReader.cpp/h</code>	纯 I/O 操作, 仅文件读取时使用
<code>c2cdist.cpp/h</code>	后分类步骤, 计算量 $O(N)$, 无可优化空间
<code>vec3.h</code>	轻量工具类
<code>point_cloud.cpp/h</code>	数据容器 + 单次 $O(N)$ 遍历

2. 性能瓶颈逐项分析

瓶颈 #1: 模拟迭代循环 — 最大性能瓶颈

位置: `CSF.cpp:159-168` → `Cloth::timeStep()` (`Cloth.cpp:97-125`)

问题分析:

```
for (i = 0; i < 500; i++) {           // 最多500次外层迭代
    cloth.timeStep();                  // 每次迭代内部:
    └─ 粒子积分 (O(P))                 // P = 粒子总数
    └─ 约束满足 (O(P × K))             // K = 每个粒子邻居数 (约8-12)
    └─ 位移统计 (O(P))
    cloth.terrCollision();              // O(P)
}
```

总计算量: $O(\text{iterations} \times P \times K)$, 其中:

- `iterations` 默认 500
- $P = \text{width_num} \times \text{height_num}$, 与 `cloth_resolution` 成反比
- $K \approx 8-12$ (近邻+次近邻约束数)

示例估算: 对于 $1000\text{m} \times 1000\text{m}$ 区域, `cloth_resolution = 0.5`:

- $P = 2000 \times 2000 = 4,000,000$ 粒子
- 每次 `timeStep` 约 $4\text{M} \times 10 = 40\text{M}$ 次约束计算
- 500 次迭代 = 20 亿次约束计算

关键问题:

- `satisfyConstraintSelf` 中对邻居列表的遍历是随机内存访问模式 (`neighborsList` 是指针数组), 缓存命中率低
- 约束校正只修改 Y 分量, 但 `Vec3` 操作涉及全部3个分量的计算

瓶颈 #2：约束满足中的数据依赖与伪并行

位置： `Particle.cpp:32-57`

问题分析：

```
// Cloth.cpp:109-111
#pragma omp parallel for
for (int j = 0; j < particleCount; j++) {
    particles[j].satisfyConstraintSelf(constraint_iterations);
}
```

虽然使用了 OpenMP 并行，但存在竞态条件：

- 粒子 A 修改自身位置后，粒子 B（A 的邻居）可能同时读取 A 的旧/新位置
- 当前代码未做同步保护，结果依赖于线程调度，具有不确定性
- 这种“先到先改”的 Gauss-Seidel 风格在串行时收敛快，但并行时牺牲了收敛精度

影响：并行版本可能需要更多迭代才能收敛，部分抵消了并行加速效果。

瓶颈 #3：光栅化空洞填充

位置： `Rasterization.cpp:22-145`

问题分析：

1. 扫描线搜索 (`findHeightValByScanline`, 第22-55行):

- 最坏情况遍历整行/整列： $O(W + H)$
- 对大量空洞粒子，总复杂度可达 $O(P \times (W + H))$

2. BFS邻居搜索 (`findHeightValByNeighbor`, 第58-100行):

- 扫描线全部失败时触发
- 可能遍历大量粒子
- 该函数未并行化

3. 高度值填充循环（第136-145行）：

- 注意：代码中 OpenMP 并行被注释掉了

```
// #ifdef CSF_USE_OPENMP
// #pragma omp parallel for
// #endif
```

这意味着空洞填充是纯串行的

瓶颈 #4：后处理（坡度平滑）

位置： `Cloth.cpp:149-371`

问题分析：

- `movableFilter` 使用 BFS 做连通分量检测，复杂度 $O(P)$
- 内部对每个连通分量调用 `findUnmovablePoint` 和 `handle_slop_connected`
- `findUnmovablePoint` 对连通区域中每个粒子检查4邻域： $O(\text{connected_size} \times 4)$
- 完全串行，无任何并行化
- 大量使用 `std::queue` 和 `std::vector` 动态分配，内存碎片化严重
- `neibors` 向量是 `vector<vector<int>>`，每次BFS都重新分配

瓶颈 #5：布料初始化的内存开销

位置： `Cloth.cpp:22-95`

问题分析：

- 每个粒子包含 `vector<Particle*>` 邻居列表，约 8-12 个指针
- 每个粒子还包含 `vector<int> correspondingLidarPointList`
- `Particle` 对象大小约 200+ 字节（含 `vector` 开销）
- 400万粒子 $\times$ 200 字节 $\approx$ 800 MB 内存
- 构造函数中双重循环建立约束，无并行化

瓶颈 #6：距离分类中的双线性插值

位置： `c2cdist.cpp:22-61`

问题分析：

- 每个点需要查找4个粒子做插值： $O(N)$ ， N 为点数
- 无 OpenMP 并行
- `groundIndexes` 和 `offGroundIndexes` 使用 `push_back`，可能频繁重分配

3. 性能瓶颈严重程度排序

排名	瓶颈	严重程度	影响范围	是否已并行
1	模拟迭代循环（约束满足）	★★★★★	全局	部分（有竞态）
2	光栅化空洞填充	★★★★☆☆	空洞多时	否（被注释）
3	后处理坡度平滑	★★★★☆☆	大连通区域	否
4	约束并行竞态	★★☆☆☆☆	精度	是（但不安全）
5	内存开销	★★☆☆☆☆	大场景	N/A
6	分类步骤	★☆☆☆☆	全局	否

4. 优化建议

4.1 高优先级优化

1. 约束满足改为 **Jacobi** 风格：

- 每轮先计算所有校正量，再统一应用
- 消除竞态条件，OpenMP 并行安全
- 代价：可能需增加迭代次数

2. 取消空洞填充的 **OpenMP** 注释：

- `Rasterization.cpp:133-135` 的并行代码被注释掉，恢复即可
- 需确保 `findHeightValByScanline` 线程安全（当前使用 `isvisited` 可能冲突）

3. 粒子数据结构重构为 **SoA**（**Structure of Arrays**）：

- 将位置、速度、加速度分离为独立数组
- 大幅提升缓存命中率，尤其是约束满足阶段的随机访问

4.2 中优先级优化

4. 后处理并行化：

- 连通分量之间天然独立，可并行
- 预分配内存避免频繁 `push_back`

5. `c2cdist` 分类并行化：

- 对点云的遍历天然可并行
- 使用预分配数组代替动态 `push_back`

6. 早停阈值调优：

- 当前 `maxDiff < 0.005` 的阈值可能过于严格
- 对于低精度场景，适当放大可显著减少迭代次数

4.3 低优先级优化

7. GPU 加速：

- Verlet 积分和约束满足天然适合 GPU 并行
- 可使用 CUDA/OpenCL 实现
- 预期加速 10-50x

8. 空间分区处理：

- 对超大场景分块处理，降低单次内存峰值
- 块间需处理边界约束

5. 核心代码行数统计

文件	行数	核心度	说明
<code>Cloth.cpp</code>	428	★★★★★	模拟核心+后处理
<code>Particle.cpp</code>	58	★★★★★	物理计算内核
<code>Rasterization.cpp</code>	146	★★★★☆	光栅化
<code>CSF.cpp</code>	212	★★★☆☆	流程编排
<code>c2cdist.cpp</code>	61	★★☆☆☆	分类
<code>Particle.h</code>	170	★★★★☆	数据结构定义
<code>Cloth.h</code>	165	★★★★☆	布料类定义

文件	行数	核心度	说明
Constraint.cpp/h	89	☆☆☆☆☆	未使用的遗留代码
核心代码合计	~632行		Cloth.cpp + Particle.cpp + Particle.h

结论：真正的核心代码集中在 Cloth.cpp（428行）和 Particle.cpp（58行）两个文件中，加上 Particle.h 的数据结构定义共约632行。这是性能优化的主要目标。